Project: Amazon Stock Price Prediction using LSTM

Project Overview

In this project, I built a **time series forecasting model to predict Amazon stock prices using deep learning**.

The objective of the project was to **analyze historical stock data and predict the future stock price movement using an LSTM neural network**.

Stock price prediction is a classic **time series forecasting problem**, where past historical data is used to forecast future trends.

Data Collection

The first step of the project was **collecting historical stock market data**.

For this purpose, I used the **yfinance API**, which allows downloading financial market data directly from Yahoo Finance.

The dataset included several important stock market features such as:

- Open price
- High price
- Low price
- Close price
- Adjusted close price
- Volume

The data was collected for **Amazon stock from 2019 to 2024**.

Exploratory Data Analysis

After collecting the data, I performed **exploratory data analysis to understand stock price behavior**.

Several visualizations were created such as:

- line plots of stock prices
- candlestick charts
- OHLC charts
- volume analysis

These visualizations helped analyze **price trends, volatility, and market behavior over time**.

Feature Engineering

Next, I performed **feature engineering by adding technical indicators** that are commonly used in financial analysis.

Relative Strength Index (RSI)

RSI measures the **momentum of price movements** and helps identify whether a stock is **overbought or oversold**.

Two RSI indicators were calculated:

- RSI with period 15
- RSI with period 30

Exponential Moving Averages (EMA)

To capture long-term and short-term trends, I calculated multiple EMAs:

- EMA with period 20 (short-term trend)
- EMA with period 100 (medium-term trend)
- EMA with period 200 (long-term trend)

These indicators help the model understand **trend direction in the stock market**.

Target Variable Creation

After feature engineering, I created **target variables for prediction**.

Two targets were defined:

Target

Difference between:

Adjusted Close Price − Opening Price

This helps measure **intraday price movement**.

TargetNextClose

This represents the **next day's closing price**, which the model tries to predict.

This was created by **shifting the adjusted close price by one day forward**.

Time Series Analysis

Since stock prediction is a **time series problem**, I performed time series analysis.

Seasonal Decomposition

The data was decomposed into three components:

- Trend
- Seasonality
- Residual noise

This helps understand underlying patterns in the stock data.

Stationarity Test

I applied the **Augmented Dickey Fuller (ADF) test** to check whether the time series was stationary.

Stationarity is important for time series modeling because many models assume that statistical properties remain constant over time.

Data Preprocessing

Before training the model, several preprocessing steps were performed.

Handling Missing Values

Rows containing missing values were removed.

Feature Selection

Some unnecessary columns like **Volume and Close** were removed to simplify the dataset.

Train-Test Split

The dataset was split into:

- 80% training data
- 20% testing data

This ensures the model is evaluated on unseen data.

Data Scaling

Because neural networks are sensitive to feature scale, I used **StandardScaler** to normalize the data.

Scaling ensures that all features are on the **same numerical range**, which improves model convergence.

Creating Time Series Sequences

For LSTM models, input data must be structured as **sequences of past observations**.

So I created sliding windows where:

- past **14 time steps** were used as input
- the model predicts the **next closing price**

This allows the model to learn **temporal patterns in stock prices**.

Model Architecture

For prediction, I used a **Long Short Term Memory (LSTM) neural network**, which is designed for sequential and time series data.

The architecture consisted of:

- First LSTM layer with 150 units
- Second LSTM layer with 32 units
- Dense layers for final prediction

LSTM networks are effective because they can **capture long-term dependencies in time series data**.

Model Training

The model was trained using:

- **20 training epochs**
- **batch size of 64**
- validation split of 10%

During training, the model learned to **capture temporal relationships between historical stock features and future price movement**.

Model Evaluation

After training, the model was evaluated on the **test dataset**.

The predicted stock prices were compared with the **actual stock prices**.

Visualization plots were created to compare:

- predicted values
- actual values

This helped analyze how well the model captured the stock price trend.

Challenges Faced

During the project, a few challenges were encountered.

Market Noise

Stock prices are highly volatile and influenced by external factors like news, macroeconomics, and investor sentiment.

This makes prediction difficult.

Non-Stationary Data

Financial time series often have changing statistical properties.

To address this, stationarity tests and transformations were applied.

Feature Scaling

Deep learning models require normalized data, so scaling was necessary for better training performance.

Future Improvements

The system can be improved by:

- using additional financial indicators
- integrating sentiment analysis from financial news
- experimenting with transformer based time series models
- using larger historical datasets

Short Ending Line

“This project helped me understand how to apply deep learning techniques such as LSTM for time series forecasting, along with feature engineering using financial indicators and building a complete end-to-end prediction pipeline.”